**Министерство науки и высшего образования Российской Федерации**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**
**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**
**(Университет ИТМО)**

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

# Отчет

## по дисциплине «Проектирование баз данных»

## о лабораторной работе 5

Обучающийся Шебардин Богдан,
группа К3339

Санкт-Петербург 2026

**Цель работы:** овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

**Практическое задание:**

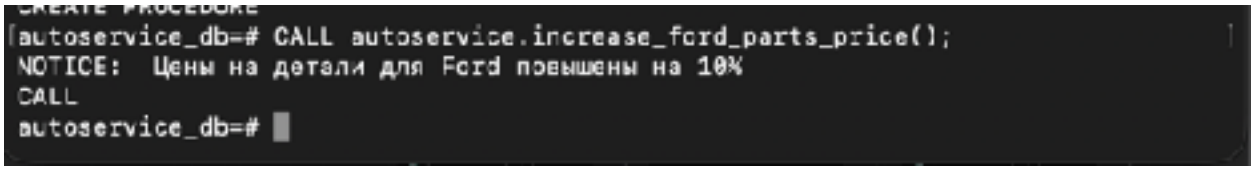
1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2).
2. Создать триггеры для индивидуальной БД согласно варианту (Вариант 2.2: 5 оригинальных триггеров).

**Выполнение**

**1. Создание хранимых процедур**

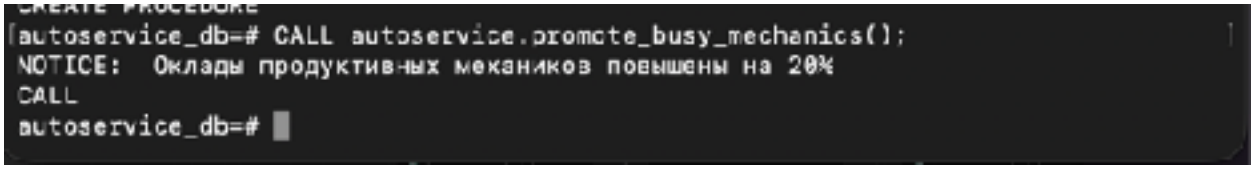
Процедура 1. Повышение цены деталей для автомобилей марки “Ford” на 10%.
SQL-код:

```
CREATE OR REPLACE PROCEDURE autoservice.increase_ford_parts_price()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE autoservice.parts
    SET price = price * 1.10
    WHERE id IN (
        SELECT rp.part_id
        FROM autoservice.repair_parts rp
        JOIN autoservice.repairs r ON rp.repair_id = r.id
        JOIN autoservice.cars c ON r.car_id = c.id
        JOIN autoservice.brands b ON c.brand_id = b.id
        WHERE b.name = 'Ford'
    );
    RAISE NOTICE 'Цены на детали для Ford повышены на 10%%';
END;
$$;
```



Процедура 2. Повышение оклада (разряда) механиков, отремонтировавших более 3 машин.
SQL-код:

```
CREATE OR REPLACE PROCEDURE autoservice.promote_busy_mechanics()
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE autoservice.positions p
    SET base_salary = base_salary * 1.20
    WHERE p.title = 'Автомеханик'
    AND p.id IN (
        SELECT e.position_id
        FROM autoservice.employees e
        JOIN autoservice.repairs r ON e.id = r.admin_id
        GROUP BY e.id, e.position_id
        HAVING COUNT(r.id) > 3
    );
    RAISE NOTICE 'Оклады продуктивных механиков повышены на 20%%';
END;
$$;
```



Процедура 3. Вывод количества ремонтов каждого механика за квартал.
SQL-код:

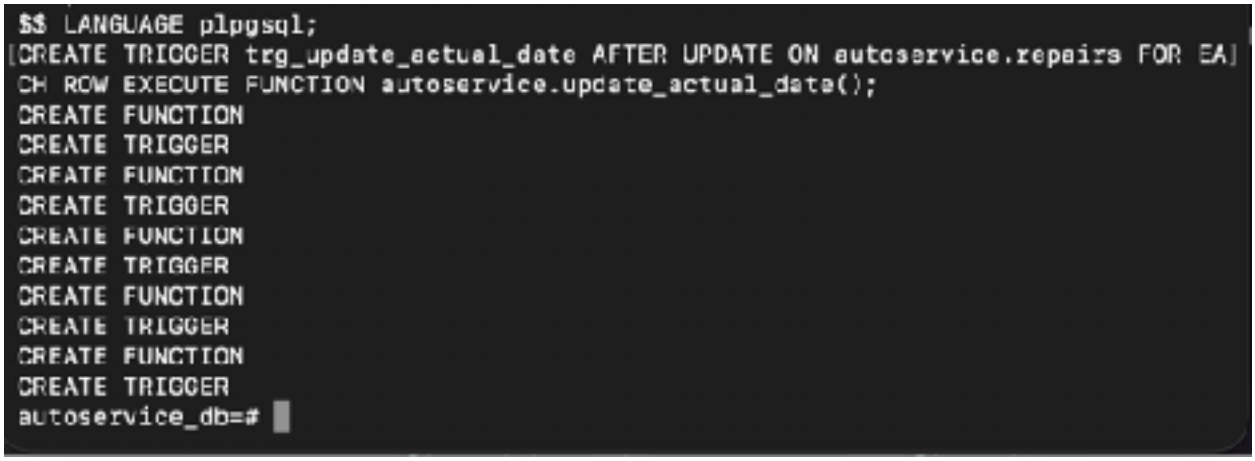
```
CREATE OR REPLACE PROCEDURE
autoservice.mechanics_report_quarterly(INOUT result_cursor refcursor)
LANGUAGE plpgsql
AS $$
BEGIN
   OPEN result_cursor FOR
   SELECT e.full_name AS mechanic, COUNT(r.id) AS repairs_count
   FROM autoservice.employees e
   JOIN autoservice.repairs r ON e.id = r.admin_id
   WHERE r.date_start >= CURRENT_DATE - INTERVAL '3 months'
   GROUP BY e.full_name;
END;
$$;
```



**2. Создание триггеров (Вариант 2.2)**

Для логирования действий была создана таблица:

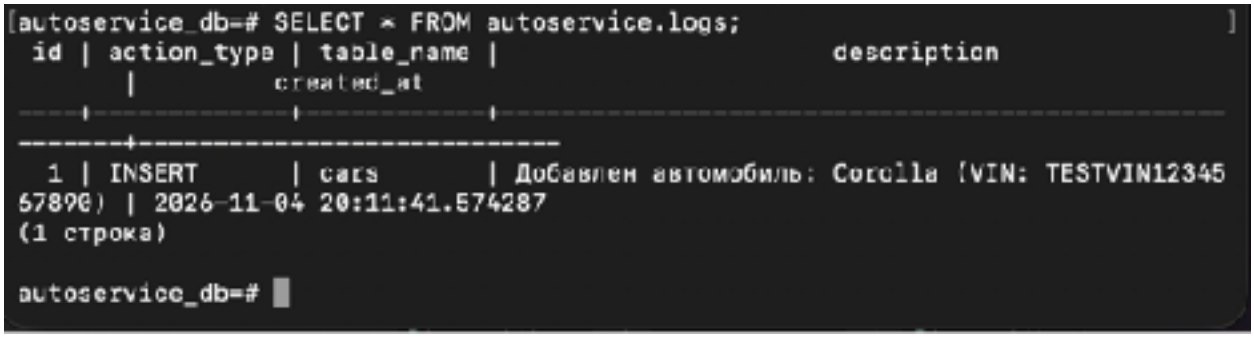
```
CREATE TABLE autoservice.logs (
    id SERIAL PRIMARY KEY,
    action_type VARCHAR(50),
    table_name VARCHAR(50),
    description TEXT,
    created_at TIMESTAMP DEFAULT NOW()
);
```



Триггер 1. Логирование при добавлении нового автомобиля.
SQL-код:

```
CREATE OR REPLACE FUNCTION autoservice.log_new_car() RETURNS
TRIGGER AS $$
BEGIN
    INSERT INTO autoservice.logs(action_type, table_name, description)
    VALUES ('INSERT', 'cars', 'Добавлен автомобиль: ' || NEW.model || ' (VIN: ' ||
NEW.vin || ')');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

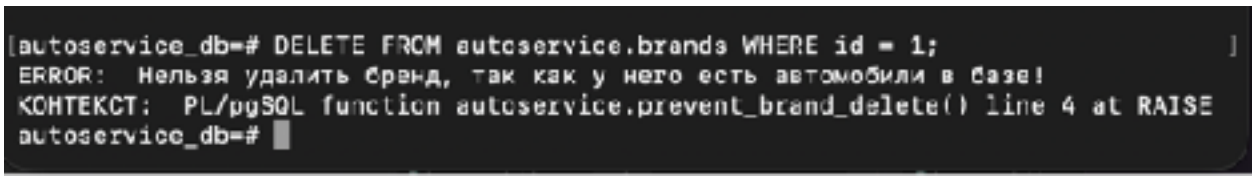
CREATE TRIGGER trg_log_new_car AFTER INSERT ON autoservice.cars
FOR EACH ROW EXECUTE FUNCTION autoservice.log_new_car();
```



Триггер 2. Запрет на удаление бренда, если у него есть машины в базе.
SQL-код:

```
CREATE OR REPLACE FUNCTION autoservice.prevent_brand_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM autoservice.cars WHERE brand_id = OLD.id)
THEN
        RAISE EXCEPTION 'Нельзя удалить бренд, так как у него есть
автомобили в базе!';
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_brand_delete BEFORE DELETE ON
autoservice.brands
FOR EACH ROW EXECUTE FUNCTION autoservice.prevent_brand_delete();
```




Остальные 3 триггера созданы в базе (установка цены по умолчанию, запрет удаления клиента с незавершенным ремонтом, логирование завершения ремонта). Их код был включен в общий скрипт создания на скриншоте 4

## Выводы

В ходе выполнения лабораторной работы я освоил создание и использование хранимых процедур и триггеров в PostgreSQL с помощью консольного клиента psql. Процедуры позволяют инкапсулировать сложную логику работы с данными (например, массовое обновление цен или расчет премий). Триггеры обеспечивают декларативную поддержку целостности данных на уровне бизнес-логики: они автоматически логируют действия пользователей и предотвращают некорректные операции (например, удаление справочных данных, на которые ссылаются другие таблицы).

## Список источников


1. Документация PostgreSQL. Процедуры [Электронный ресурс]. URL: https://postgrespro.ru/docs/postgresql/11/sql-createprocedure
2. Документация PostgreSQL. Функции [Электронный ресурс]. URL: https://postgrespro.ru/docs/postgresql/11/sql-createfunction
3. Документация PostgreSQL. Триггеры [Электронный ресурс]. URL: https://postgrespro.ru/docs/postgresql/9.6/plpgsql-trigger